Los Alamos NATIONAL LABORATORY

# Continuous Performance Validation at LANL: Integrating Hardware, Software, and Application-Level Testing

Paul Ferrell, Lena Lopatina, Carola Ellinger,
Hank Wikle, Shivam Mehta, Andres Ortiz

4/25/26

LA-UR-26-23267

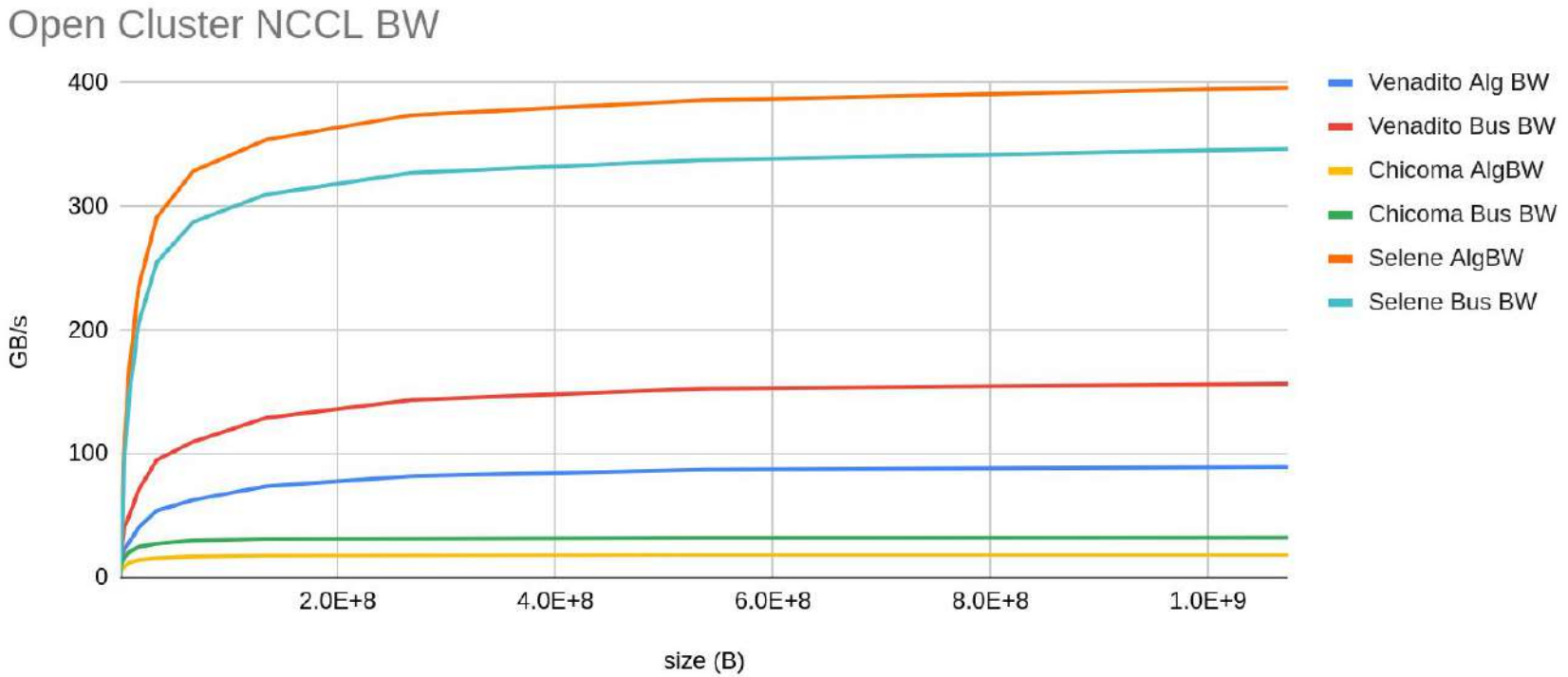
# Pavilion2

- Open Source
  - https://github.com/hpc/pavilion2
- YAML configuration based
- Designed for writing tests generically to run on significantly different clusters.
- Advanced test scheduling features (Slurm, FLUX, PBS (soon))
  - System coverage testing (chunking)
  - Multiple tests per job
  - Asynchronous tests per job

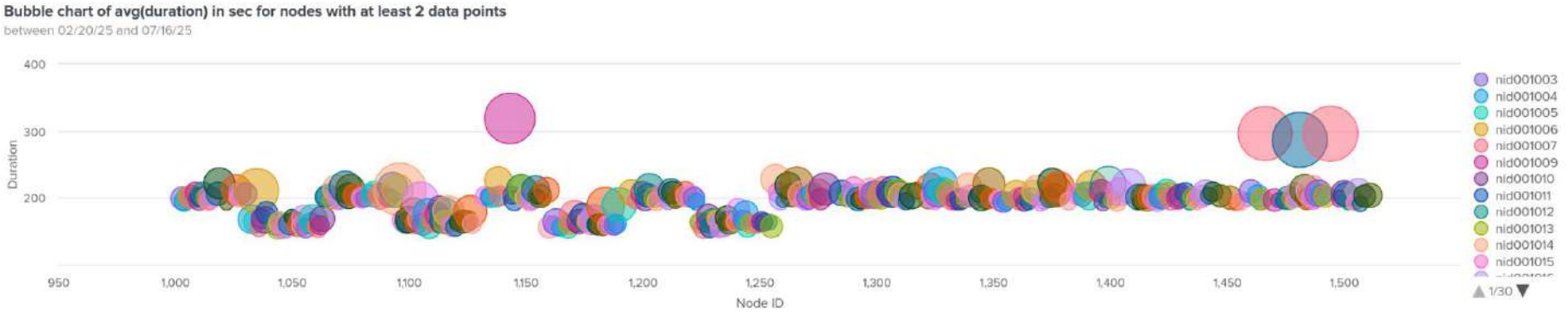
# Major Changes/Additions

- Improved CI Support
- Result handler plugins
- Test runs now have series-relative IDs
  - IE Test # ID's '23452', '23452', … -> 's3.1', 's3.2', …
- Improved Test Organization
  - Tests are fully encapsulated as a 'suite' directory
  - Per-suite host and mode configurations
- New features to support debugging tests
  - Test builds and runs now output their environments for easy inspection
  - Ability to quickly isolate tests from Pavilion via `pav isolate`

# Nvidia GPU testing



# Continuous Testing



# Continuous Testing

# Continuous Testing

# Application Performance Testing: Overview

PerfEng team at LANL works with multiple code teams to develop continuous performance testing and benchmarking:

- each team designates liaisons who work with host teams to identify representative user problems to be monitored
- regular test cadence (nightly, and weekly for larger problems in future)
- currently we are collecting performance data for :
  - multiple HPC clusters, multiple builds variants
  - total run time and throughput; runtime per package; high water memory watermark
  - weak- and strong- scaling results
  - provide host teams with:
    - one-page e-mail summary of daily results
    - interactive Splunk dashboards containing all raw and aggregated data

# Application Performance Testing: Success and Challenges

**Success:**

- last year our benchmarks began reporting a large number of failures
- we traced those failures to out-of-memory (OOM) events
- using the nightly data series we narrowed the onset of OOMs to a specific time window
- using an automated script we pinpointed the exact Merge Request responsible for the increased memory usage
- a newly-added compiler flag raised the memory footprint
- after weighing the pros and cons, the code team reverted that compiler flag, restoring the original lower-memory footprint

# Application Performance Testing: Success and Challenges

**Challenges:**

- at that time we had intermittent test results, that is why we had to use a script to bisect merge requests (MRs) and identify which one affected the memory footprint
- the tool that reported the memory footprint actually showed a **decrease** in memory usage, which was misleading
  - that is why now we run tests close to the maximum memory limit
  - we are working on identifying and developing additional memory tracking approaches and tools

# Application Performance Testing: Future work

**Continuous work:**

- it is extremely important to work closely with host teams on identifying problems of interest as well as providing host teams with high-value information they can act on.

**Future work:**

- combine data sources—merge application-performance metrics with system-side monitoring data—to achieve holistic monitoring and enable early, automated detection of anomalies.

# Questions